

Entrega 4 - Hola mundo

Ángel David Suescun Romero
asuescunr@unal.edu.co

Laura Camila Perilla Torres
lperillat@unal.edu.co

Juan Pablo Rodríguez Briceño
jrodirguezbr@unal.edu.co

Wilson Franco Martinez
wfrancom@unal.edu.co

Asignatura: Ingeniería de software I

Docente: Oscar Eduardo Alvarez Rodriguez

Universidad Nacional de Colombia
Sede Bogotá

Bogotá D.C
Colombia

TABLA DE CONTENIDOS

Tecnologías usadas.....	3
- Instalación de Java.....	3
- Instalación de Docker.....	3
Creación del proyecto.....	4
- Manejo de dependencias.....	5
Ejemplo de proyecto.....	6
- User.java.....	6
- UserRepository.java.....	7
- UsuarioService.java.....	7
- UserController.java.....	8
Creación de Base de Datos en MySQL.....	11
Creación del contenedor con Docker.....	12
Peticiones HTTP.....	13
- GET: Todos los usuarios.....	13
- GET: Usuario por ID.....	14
- POST: Registrar usuario.....	14
- PUT: Actualizar usuario.....	14
- DELETE: Eliminar usuario.....	15

Tecnologías usadas

En este tutorial se usarán las siguientes tecnologías para el desarrollo de la aplicación:

- Java 21
- Spring Boot Framework
- Hibernate - Spring Data JPA (ORM)
 - Hibernate implementa el patrón ORM (Object Relational Mapping) lo cual permite representar las tablas de la base de datos mediante clases de Java
- Base de Datos MySQL
- Gestor de dependencias Maven
- Contenedores (Docker)

- Instalación de Java

Antes de instalar java, se puede correr el siguiente comando en alguna terminal o en PowerShell:

```
java -version
```

Si el dispositivo a usar lo tiene instalado, saldrá algo similar a:

```
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode,
sharing)
```

De no ser así, se debe descargar la versión de Java de acuerdo al sistema operativo a través de este enlace:

<https://www.oracle.com/java/technologies/downloads/>

- Instalación de Docker

Antes de instalar Docker, se puede correr el siguiente comando en alguna terminal o en PowerShell:

```
Docker --version
```

Si el dispositivo a usar lo tiene instalado, saldrá algo similar a:

```
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
```

De no ser así, se debe descargar el motor de Docker respectivo a la versión de linux a través de este enlace:

<https://www.oracle.com/java/technologies/downloads/>

Si se busca realizar la instalación para Windows o macOS, es necesario realizar primero la instalación de Docker Desktop. Se puede encontrar más información a través del siguiente enlace:

<https://docs.docker.com/desktop/>

Creación del proyecto

Teniendo en cuenta que queremos crear un nuevo proyecto con spring, podemos hacer uso de un generador de proyectos Spring Boot. Se puede acceder al enlace <https://start.spring.io/> para realizar esto.

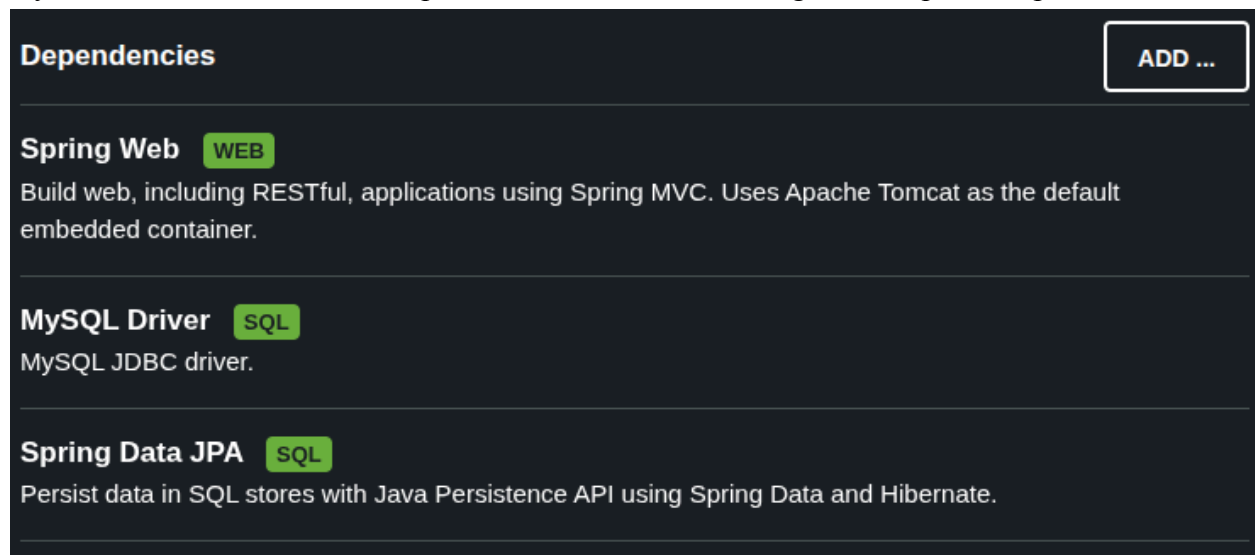
Las opciones que se ven en la imagen son las recomendadas para el proyecto, con la excepción de la versión de Java, la cual debe coincidir con la versión de Java que se haya instalado.



The image shows the Spring Initializr web form with the following configuration:

- Project:** ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Maven
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 4.1.0 (SNAPSHOT), ☐ 4.1.0 (RC1), ☐ 4.0.7 (SNAPSHOT), ☒ 4.0.6, ☐ 3.5.15 (SNAPSHOT), ☐ 3.5.14
- Project Metadata:**
 - Group: com.example
 - Artifact: helloworld
 - Package name: com.example.helloworld
 - Packaging: ☒ Jar, ☐ War
 - Configuration: ☒ Properties, ☐ YAML
 - Java: ☐ 26, ☐ 25, ☐ 21, ☒ 17

Una vez hayamos seleccionado las características del proyecto, procedemos a agregar las dependencias necesarias. Teniendo en cuenta que se quiere tener un proyecto API REST que use MySQL, se deben instalar las dependencias indicadas en la siguiente captura de pantalla:



Entre las dependencias seleccionadas se hablará un poco de Spring Data JPA. Esta dependencia hace uso de la librería Hibernate, la cual es una librería ORM (Object-Relational Mapping) que permite una conexión entre la base de datos y la aplicación de una manera más amigable para el desarrollador, ya que evita que tengamos que manipular la base de datos a través de la aplicación usando cadenas que representen instrucciones SQL.

Una vez tenemos las propiedades y las dependencias del proyecto, podemos proceder a Generar el proyecto. Esto hará que la herramienta descargue un archivo con el nombre indicado en el campo de “Artifact” (De nombre “helloworld.zip” para este ejemplo).

Procedemos a descomprimir este archivo .zip en alguna carpeta de nuestro ordenador en el que tendremos el proyecto. Es importante remarcar que toda la información de las dependencias va a ser almacenada en el archivo pom.xml.

- Manejo de dependencias

Si se busca agregar, remover o modificar dependencias en un futuro, se debe modificar, agregar o eliminar el archivo a partir de la línea `<dependencies>`. Si se modifica este archivo, es importante ejecutar el siguiente comando para aplicar los cambios en las dependencias del proyecto:

```
mvn clean install
```

Ejemplo de proyecto

Por efectos de practicidad, se ponen varios archivos que deben ser incluidos en el proyecto:

- User.java

Este archivo está encargado de representar a los usuarios como objetos dentro de la aplicación.

```
package com.example.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "user")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    // Constructores
    public User() {}

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Getters y Setters
```

```

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}

```

Ruta del archivo: src/main/java/com/example/domain/

- UserRepository.java

Esta clase es la que permite que se pueda interpretar la información de las tablas de la base de datos con las entidades creadas en el proyecto. Es una clase corta, ya que todo el trabajo se logra por la extensión de JpaRepository.

```

package com.example.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.example.domain.User;

@Repository

```

```
public interface UserRepository extends JpaRepository<User, Long> {  
}
```

Ruta del archivo: src/main/java/com/example/repository/

- **UsuarioService.java**

Esta clase es la que se encarga de contener y procesar toda la lógica de negocio detrás del proyecto, así que esta puede ser una clase muy larga dependiendo de las necesidades de cada entidad.

```
package com.example.service;  
  
import java.util.List;  
import java.util.Optional;  
  
import org.springframework.stereotype.Service;  
  
import com.example.domain.User;  
import com.example.repository.UserRepository;  
  
@Service  
public class UserService {  
  
    private final UserRepository repository;  
  
    public UserService(UserRepository userRepository) {  
        this.repository = userRepository;  
    }  
  
    public List<User> findAll() {  
        return repository.findAll();  
    }  
  
    public Optional<User> findById(Long id) {  
        return repository.findById(id);  
    }  
  
    public User save(User u) {  
        return repository.save(u);  
    }  
}
```



```

    public User update(Long id, User userDetails) {
        return repository.findById(id)
            .map(user -> {
                user.setName(userDetails.getName());
                user.setEmail(userDetails.getEmail());
                return repository.save(user);
            })
            .orElseThrow(() -> new RuntimeException("Usuario no
encontrado con id: " + id));
    }

    public void delete(Long id) {
        if (!repository.existsById(id)) {
            throw new RuntimeException("Usuario no encontrado con id:
" + id);
        }
        repository.deleteById(id);
    }

    public User findFirst() {
        return
repository.findAll().stream().findFirst().orElse(null);
    }
}

```

Ruta del archivo: src/main/java/com/example/service/

- UserController.java

Esta clase es la que permite que algún cliente pueda acceder a la información de la aplicación y poder crear, actualizar, consultar o borrar información de acuerdo a las funciones especificadas en el Service.

En este archivo se le puede indicar a cada función el cómo se debe activar de acuerdo a los contenidos de la URL y el tipo de petición.

```

package com.example.controller;

import java.util.List;

```

```
import java.util.Optional;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.example.domain.User;
import com.example.service.UserService;

@RestController
@RequestMapping("/user")
@CrossOrigin(origins = "*")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    /**
     * GET /user - Obtiene todos los usuarios
     */
    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        List<User> users = userService.findAll();
        return ResponseEntity.ok(users);
    }

    /**
     * GET /user/{id} - Obtiene un usuario por ID
     */
}
```

```

    */
    @GetMapping("/{id}")
    public ResponseEntity<?> getUserById(@PathVariable Long id) {
        Optional<User> user = userService.findById(id);
        if (user.isPresent()) {
            return ResponseEntity.ok(user.get());
        }
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body("Usuario no encontrado con id: " + id);
    }

    /**
     * POST /user - Crea un nuevo usuario
     */
    @PostMapping("/")
    public ResponseEntity<?> createUser(@RequestParam String name,
        @RequestParam String email) {
        try {
            User user = new User();
            user.setName(name);
            user.setEmail(email);

            if (user.getName() == null ||
user.getName().trim().isEmpty()) {
                return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body("El nombre es requerido");
            }
            if (user.getEmail() == null ||
user.getEmail().trim().isEmpty()) {
                return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body("El email es requerido");
            }

            User createdUser = userService.save(user);
            return
ResponseEntity.status(HttpStatus.CREATED).body(createdUser);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body("Error al crear usuario: " +

```

```

e.getMessage());
    }
}

/**
 * PUT /user/{id} - Actualiza un usuario existente
 */
@PutMapping("/{id}")
public ResponseEntity<?> updateUser(@PathVariable Long id,
@RequestParam String name, @RequestParam String email) {
    try {
        User userDetails = new User();
        Optional<User> user = userService.findById(id);
        if (!(name==null) && !(name.trim().isEmpty())) {
            userDetails.setName(name);
        }
        if (!(email==null) && !(email.trim().isEmpty())) {
            userDetails.setEmail(email);
        }

        User updatedUser = userService.update(id, userDetails);
        return ResponseEntity.ok(updatedUser);
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(e.getMessage());
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body("Error al actualizar usuario: " +
e.getMessage());
    }
}

/**
 * DELETE /user/{id} - Elimina un usuario
 */
@DeleteMapping("/{id}")
public ResponseEntity<?> deleteUser(@PathVariable Long id) {
    try {
        userService.delete(id);
    }
}

```

```

        return ResponseEntity.ok("Usuario eliminado
correctamente");
    } catch (RuntimeException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(e.getMessage());
    } catch (Exception e) {
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Error al eliminar usuario: " +
e.getMessage());
    }
}
}
}

```

Ruta del archivo: src/main/java/com/controller/

Creación de Base de Datos en MySQL

Se puede trabajar una base de datos y generar el archivo .sql que permita replicar dicha base de datos. Este archivo debe llamarse init.sql y estar ubicado en la carpeta raíz del proyecto.

```

-- init.sql
-- Este archivo se ejecuta automáticamente al iniciar el contenedor
MySQL

USE holamundo_db;

-- Crear tabla de usuarios (Hibernate también la puede crear, pero
aquí explícitamente)
CREATE TABLE IF NOT EXISTS usuarios (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
);

-- Insertar datos de ejemplo (opcional)
INSERT INTO usuarios (nombre, email) VALUES
('Juan Pérez', 'juan@example.com'),
('María García', 'maria@example.com'),

```

```
('Carlos López', 'carlos@example.com');
```

Creación del contenedor con Docker

Docker requiere de un archivo de nombre **docker-compose.yml** (el cual debe estar en la carpeta raíz del proyecto) en donde se le indica las propiedades de la base de datos y del contenedor. Para ello, facilitamos el siguiente archivo:

```
# docker-compose.yml

version: '3.8'

services:
  mysql:
    image: mysql:latest
    container_name: hw_mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: hw_db
      MYSQL_USER: user
      MYSQL_PASSWORD: user

    ports:
      - "3306:3306"

    volumes:
      - mysql_data:/var/lib/mysql
      - ./data/init.sql:/docker-entrypoint-initdb.d/01-init.sql

volumes:
  mysql_data:
```

Este archivo es necesario para que el contenedor se cree correctamente. Si se comete un error de cualquier tipo con este archivo, el contenedor se creará pero podrá generar un contenedor que no se podrá usar.

Una vez se tenga este archivo, se procede a abrir una terminal que esté ubicado en la carpeta raíz del proyecto y se ejecuta el siguiente comando:

```
docker-compose up -d
```

Este comando va a comenzar la creación del contenedor teniendo en cuenta los parámetros del archivo docker-compose.yml. Asimismo va a crear la base de datos dentro del contenedor.

Es posible que se requieran permisos de administrador para poder ejecutar este comando.

Una vez que se haya creado el contenedor, podemos probar correr nuestro proyecto de Spring Boot. Para ello, es necesario correr el siguiente comando:

```
./mvnw spring-boot:run
```

Peticiones HTTP

Cuando la aplicación esté ejecutándose, podremos usar otra herramienta como Postman, Thunderclient o cualquier otra que nos permita hacer peticiones HTTP.

A continuación se hacen ejemplos con las capacidades de este programa:

- **GET: Todos los usuarios**

URL:

```
http://localhost:8080/user
```

Parámetros: N/A

Estado: 200

Resultado:

```
[
  {
    "name": "Juan Pérez",
    "email": "juan@example.com",
    "id": 1
  },
  {
    "name": "María García",
    "email": "maria@example.com",
```

```
    "id": 2
  },
  {
    "name": "Carlos López",
    "email": "carlos@example.com",
    "id": 3
  }
]
```

- GET: Usuario por ID

URL:

```
http://localhost:8080/user/{id}
```

Parámetros:

Id: Identificador del usuario

Estado: 200

Resultado:

```
{
  "name": "Juan Pérez",
  "email": "juan@example.com",
  "id": 1
}
```

- POST: Registrar usuario

URL:

```
http://localhost:8080/user/?name={name}&email={email}
```

Estado: 201

Parámetros:

name: Nombre del usuario

email: Correo del usuario

Resultado:

```
{
  "name": "Profe Oscar",
  "email": "po@ke.mon",
  "id": 4
}
```


- **PUT: Actualizar usuario**

URL:

```
http://localhost:8080/user/{id}?name={name}&email={email}
```

Parámetros:

Id: Id del usuario a actualizar

name: Nuevo nombre del usuario

email: Nuevo correo del usuario

Estado: 200

Resultado:

```
{
  "name": "Oscar Eduardo",
  "email": "oscar@example.com",
  "id": 4
}
```

- **DELETE: Eliminar usuario**

URL:

```
http://localhost:8080/user/{id}
```

Parámetros:

Id: Id del usuario a eliminar

Estado: 200

Resultado:

```
Usuario eliminado correctamente
```